

Sparsh / Samriddhi VD641NT — protocol reference (as used by EquiCare)

Distilled from the vendor documents so the driver does not depend on an attachment in someone's inbox. Source documents, both supplied by the vendor:

- "Samriddhi Automations Network Camera — Web API Protocol Description", doc version **B22, 2025-11-25** (the ISAPI reference). *Not in this repo — request a copy from the vendor and save it beside this file.*
- "Thermal camera Modbus/TCP communication protocol" — `ModbusTCP communication protocol.pdf`, tracked in this folder.

Everything below is reachable from **ARM64** over plain HTTP / Modbus / RTSP. The vendor's C/C++/C# SDK (`IR 2.0sdk/`) is **x86-64 only** and is deliberately not used. Implementation: [`sparsh_camera.py`](../sparsh_camera.py).

1. Conventions

Convention	Value
Base path	<code>http://<ip>:80/ISAPI/</code> (HTTPS on 443 if enabled)
Content type	<code>application/json; charset=utf8</code>
Device selector	<code>Dev=0</code> infrared · <code>Dev=1</code> visible · <code>Dev=2</code> fusion
Channel selector	<code>Chn=0</code> main stream · <code>Chn=1</code> sub-stream
Temperatures	integer $^{\circ}\text{C} \times 100$ (<code>3712</code> = 37.12 $^{\circ}\text{C}$)
Coordinates	proportional, 0–10000 (<code>RatX</code> / <code>RatY</code>) — resolution-independent
Emissivity	<code>FPara100</code> = emissivity $\times$ 100, range 1–100
Distance	<code>AimDistance</code> in centimetres
HTTP verbs	<code>GET</code> read · <code>PUT</code> update · <code>POST</code> add/act

Failure body: `{ "Result": "Failed", "Reason": "...", "Code": -400, "URI": "/ISAPI/..." }`. Useful codes: `-400` unsupported, `-402` permission denied, `-424` invalid parameter, `-440` auth failed.

2. Authentication

Two-step session auth (digest MD5; SHA256 also supported):

```
PUT /ISAPI/Security/User/Login { "Realm": <random 8-char string we generate>, "Name": MD5("<user>:<ourRealm>"), "Password": MD5( MD5("<user>:<deviceRealm>:<pass>") + ":" + <ourRealm> ) }
→ { "SessionID": "<64 chars>", "Permission": "Administrator" }
```

Every later request carries the header `SessionID: <value>`. MD5 output must be **lowercase hex**.

> **Gotcha.** The machine-translated doc prints the device realm literally as > "Server Status" . The driver first tries to read the real realm from a > WWW-Authenticate header, and falls back to plain **HTTP Digest** auth if the > session login is rejected — some firmware accepts that instead.

3. Capability and identity

Endpoint	Purpose
GET /ISAPI/System/Capability/DeviceInfo	model, serial, firmware, IRModule , CCDModule
GET /ISAPI/System/Capability/CSCI	feature flags — check these before assuming support
GET /ISAPI/System/Capability/RtspStreamAddress	RTSP URLs
GET /ISAPI/System/Capability/DeviceStat	live alarm state + current ROI max temps

Flags that matter to us, from CSCI :

- WithDMMeasure — point/line/area thermometry. **Must be** Yes ; it is the marker of the temperature-measuring **NT** variant.
- WithCCD — visible channel present (dual-spectrum unit).
- WithMetaRaw — raw data / meta stream available.
- WithBlackBody — expect No . Confirms the ±2 °C screening-grade ceiling.

4. Thermometry — the core of points 2, 3, 4

```
`` GET /ISAPI/Thermometry/BasicParam?Dev=0 global emissivity, distance, ambient, unit PUT
/ISAPI/Thermometry/BasicParam?Dev=0 GET /ISAPI/Thermometry/Query?Dev=0&Type=255 live values;
Type 0=Point 1=Line 2=Area 3=Circle 255=all GET|PUT|POST /ISAPI/Thermometry/Point?Dev=0&Idx=0
Idx 0-9 (255 = get all) GET|PUT|POST /ISAPI/Thermometry/Line?Dev=0&Idx=0 Idx 0-2 GET|PUT|POST
/ISAPI/Thermometry/Area?Dev=0&Idx=0 Idx 0-9, 3-20 vertices GET|PUT|POST
/ISAPI/Thermometry/Circle?Dev=0&Idx=0 Idx 0-9, radius in pixels GET|PUT|POST
/ISAPI/Thermometry/AreaDiff?Dev=0 temperature delta between two areas PUT
/ISAPI/Thermometry/Delete?Dev=0 { "Type":"Point", "Id":0 } (Id 255 = all) ``
```

Setting a point ROI — this is what makes runtime ROI steering possible without the SDK:

```
`` json PUT /ISAPI/Thermometry/Point?Dev=0&Idx=0 { "ThermometryList": [{ "Id": 0, "PresetIdx": 0,
"Type": "Point", "Enable": "Yes", "Name": "eye", "Point": { "RatX": 5000, "RatY": 5000 },
"FPara100": 98, "AimDistance": 350, "TempAlarm": { "UpType": 0, "UpLimit": 7000, "DownType": 0,
"DownLimit": 1000 }, "AlarmFilterTime": 2, "AlarmLinkOutInfo": "", "ActiveTimeSet": "ffffff-
ffffff-ffffff-ffffff-ffffff-ffffff-ffffff-" }] } → { "Result": "OK", "URI":
"/ISAPI/Thermometry/Point" } ``
```

Query returns, per ROI: PointTemp for points; MaxTemp / MinTemp / AvgTemp for line/area/circle — each {Value, RatX, RatY} with Value in °C × 100.

ActiveTimeSet is the arming schedule: 24 bits of hours per day as hex, one field per day from Sunday, – separated. fffffff = always armed.

Practical limits (vendor-stated, not doc-stated): ROI refresh is ~20 Hz and not configurable; temperature polling is capped by the 25 Hz sensor frame rate, and the vendor recommends 1–10 Hz. The constraint is **per device, not per host**, so multiple cameras scale independently.

5. Streams

```
`` IR main rtsp://<ip>:554/stream/live?dev=0&chn=0 IR sub rtsp://<ip>:554/stream/live?dev=0&chn=1
visible rtsp://<ip>:554/stream/live?dev=1&chn=0 fusion rtsp://<ip>:554/stream/live?dev=2&chn=0
(640x512) raw rtsp://<ip>:554/stream/raw?dev=0&chn=0 temperature rtsp://<ip>:554/stream/meta?
dev=0&chn=0 ← timestamped push ``
```

Credentials may be inlined: `rtsp://user:pass@<ip>:554/...`. HTTP-FLV is also offered on port 8008 via `/ISAPI/System/Capability/HttpStreamAddress`.

5.1 THE META STREAM — BEST PATH FOR TIME ALIGNMENT

64-byte little-endian header, then a temperature JSON payload in the same schema as `Thermometry/Query`:

```
`` c struct { // 64 bytes total int32 magic; // 0x55AAAA55 int8 major, minor; // protocol 1.0
int16 frame_type; // 0x0000 = text/JSON, 0x0100 = JPEG int32 frame_length; // payload length,
header excluded int32 frame_sec; // seconds since 1970-01-01 int32 frame_msec; // 0-999 int32
group_id; // ties frames across channels for sync int32 ip_address; // 0xAABBCCDD int8
reserved[36]; } ``
```

This matters: temperatures arrive **stamped at the sensor/driver layer** with a `group_id`, rather than being correlated by host wall-clock after a poll. Prefer it over polling for respiration work. Decoder: `parse_meta_frame()`.

6. Snapshots

```
`` GET /ISAPI/Snapshot/JPG?Dev=0&Type=0 plain JPEG GET /ISAPI/Snapshot/JPG?Dev=0&Type=1 JPEG +
per-pixel radiometric data appended GET /ISAPI/Snapshot/DLV?Dev=0 vendor DLV container ``
```

`Type=1` is our route to **per-pixel** temperature — the ROI API only ever returns aggregates. Format described in `Temp data picture+picture format instruction/`.

7. Other endpoints we rely on

Endpoint	Use
<code>PUT /ISAPI/System/Time/NTP</code>	point the camera at our edge box as NTP source
<code>GET/PUT /ISAPI/Mpp/Video/EncodeParam?Dev=0&Chn=0</code>	resolution, frame rate, CBR/VBR, bitrate
<code>PUT /ISAPI/Mpp/Video/MSX</code>	fusion <code>Distance</code> in cm — set ~350 for our 3.5 m geometry
<code>PUT /ISAPI/Mpp/Video/ManualAdjust?Dev=0</code>	trigger NUC / shutter calibration
<code>GET /ISAPI/Smart/Event/Query</code>	current alarm state (we poll rather than use ONVIF events)
<code>PUT /ISAPI/System/Ctrl/Reboot</code>	recovery

8. Modbus/TCP — temperature read (port 502, function 03)

Independent of ISAPI and trivially ARM-native. Register addresses are **documented 1-based**; subtract 1 on the wire (doc 1019 → wire 0x03FA).

ROI <i>n</i> (1–10)	Point	Area min	Area max	Area avg
Base address	1019 + 12(<i>n</i> –1)	+2	+4	+6
<i>n</i> = 1	1019–1020	1021–1022	1023–1024	1025–1026
<i>n</i> = 10	1127–1128	1129–1130	1131–1132	1133–1134

Each value is a **32-bit IEEE-754 float across two registers, little-endian**: first register holds the low 16 bits. No byte-order conversion is applied in transit.

> Verified: the decoder reproduces the vendor's worked example exactly — > 0x3333,0x41C7 → 24.9 °C, 0x999A,0x41ED → 29.7 °C.

Read-only. ROIs must be *created* over ISAPI (or the web UI) first; Modbus only reports them.

9. Known limitations — designed around, not solvable here

Limitation	Consequence
No ARM64 SDK build, and the vendor has no team to produce one	We use ISAPI + Modbus + RTSP exclusively
No visible→thermal coordinate mapping exposed; fusion overlay "approximate" and error unquantified	Detect on the thermal/fusion stream; do not map from the visible frame
±2 °C accuracy, no blackbody variant (upgrade path retracted)	Screening-grade only — never present as clinical
No PTP, NTP only	Software alignment; vendor states ≤40 ms (~1 frame) achievable

Limitation	Consequence
ROI refresh ~20 Hz, not configurable	Fast nostril tracking is limited; static ROIs for resting animals
850 nm IR illuminator (not 940 nm)	Visible red glow disturbs horses at night → external 940 nm illuminators required
Rolling-shutter visible sensor (IMX290)	IMU is primary for gait/lameness; optical gait is supporting only
No ONVIF Profile G event schema available	We poll instead of ingesting events
Fixed focus built to order ; 25 mm in focus only ~3.0–4.3 m	Mounting geometry must hold ~3.5 m; state focus distance on the PO